

LUỒNG VẬN HÀNH DOCKER COMPOSE

Tài liệu này phân tích chi tiết cơ chế hoạt động tầng dưới của Docker Compose khi thực thi lệnh khởi chạy hệ thống, đồng thời cập nhật theo cấu hình thực tế sử dụng các Image pre-built được lưu trữ trên Docker Hub.

1. Giải thích chi tiết về lệnh và các Cờ cấu hình (Flags & Configuration)

Khi thực thi lệnh 'sudo docker-compose up -d --build', hệ thống sẽ phân tích các cờ và xử lý tệp cấu hình theo cơ chế sau:

1.1 Ý nghĩa của các Flag trong lệnh:

- **sudo:** Cho phép chạy lệnh dưới quyền quản trị tối cao (root). Docker Daemon chạy như một dịch vụ nền và giao tiếp qua Unix Socket '/var/run/docker.sock' - vốn chỉ cho phép root hoặc thành viên nhóm 'docker' truy cập.
- **docker-compose up:** Lệnh cốt lõi yêu cầu Docker Compose đưa toàn bộ hệ thống dịch vụ về trạng thái mong muốn (desired state). Nó tự động thực hiện tải ảnh, cấu hình mạng ảo, phân bổ volume, tạo và khởi chạy các container.
- **-d:** Detached mode. Cho phép các container chạy dưới dạng tiến trình ngầm (background processes). Docker Client sẽ ngắt kết nối stream logs sau khi khởi động thành công và trả lại quyền điều khiển cho cửa sổ Terminal.
- **--build:** Thường dùng để ép buộc Docker Compose build lại image từ local Dockerfile. Tuy nhiên, vì cấu hình hiện tại của bạn đã đổi sang sử dụng image pre-built trên Docker Hub cho cả frontend và backend, Docker Compose sẽ tự động bỏ qua bước build này. Cờ này được giữ lại trong câu lệnh mẫu để phòng trường hợp tương lai bạn chuyển lại cấu hình sang build trực tiếp từ mã nguồn.

1.2 Thứ tự đọc và xử lý cấu hình:

Docker Compose áp dụng trình tự nghiêm ngặt sau để phân tích và chuẩn bị môi trường chạy:

- **1. Biến môi trường Host OS:** Đầu tiên, đọc các biến từ shell máy chủ hiện hành.
- **2. Tệp .env:** Tìm kiếm và nạp các biến từ tệp '.env' ở thư mục chạy lệnh. Nếu một biến tồn tại ở cả Host OS và file '.env', giá trị trên Host OS sẽ được ưu tiên hơn.
- **3. Tệp cấu hình chính:** Đọc tệp docker-compose.yml và các tệp override cấu hình (ví dụ: docker-compose.override.yml nếu có).
- **4. Nội suy biến (Variable Interpolation):** Thay thế các cú pháp dạng \${VARIABLE} hoặc \$VARIABLE bằng giá trị cụ thể đã nạp ở bước trước.
- **5. Hợp nhất cấu hình (Merged Config):** Hợp nhất toàn bộ cấu hình thành một đặc tả hệ thống dạng JSON để chuyển qua REST API cho Docker Daemon xử lý.

2. Luồng hoạt động và Thứ tự khởi chạy Container (Build & Deployment Workflow)

2.1 Tương tác giữa Docker Client và Docker Daemon (Engine):

Docker Client và Daemon tương tác qua REST API trên Socket `"/var/run/docker.sock"`. Client chịu trách nhiệm phân tích tệp compose và gửi lệnh, còn Daemon thực hiện mọi thao tác tải ảnh, tạo mạng ảo, volume, và điều khiển vòng đời container.

2.2 Luồng dữ liệu: Quy trình Đóng gói (Build & Push) và Triển khai (Pull & Run):

Vì hệ thống sử dụng các image đóng gói sẵn trên Docker Hub, luồng dữ liệu trải qua các giai đoạn lớn sau:

- **Giai đoạn 1: Đóng gói & Phát hành (Developer):** Lập trình viên viết mã nguồn, viết Dockerfile và thực hiện dựng (build) các image `'vokhoecho/pharmacy-frontend:2.0'` và `'dinhsi1401/pharmacy:2.0'` ngay trên máy của mình. Sau đó, đẩy (push) các image này lên kho lưu trữ Docker Hub.
- **Giai đoạn 2: Tải ảnh (DevOps Pull):** Khi DevOps chạy lệnh triển khai trên máy chủ, Docker Daemon sẽ liên hệ Docker Hub để kéo (pull) các image này về máy chủ nếu chưa có sẵn bản mới nhất.
- **Giai đoạn 3: So sánh & Tái tạo (Reconciliation):** Docker Daemon tiến hành đối chiếu cấu hình mong muốn trong tệp `docker-compose.yml` với trạng thái các container cũ đang chạy. Nếu phát hiện sự khác biệt hoặc có Image ID mới, container cũ sẽ bị dừng, xóa bỏ và container mới sẽ được khởi tạo từ image vừa tải.

2.3 Cơ chế quyết định thứ tự khởi chạy Container:

Docker Compose thiết lập một Đồ thị chỉ hướng không chu trình (Directed Acyclic Graph - DAG) dựa trên thuộc tính `'depends_on'` khai báo trong tệp cấu hình:

- **1. mongodb:** Dịch vụ database mongodb không phụ thuộc bất kỳ ai, do đó nó được tạo và khởi chạy đầu tiên.
- **2. spring-boot:** Dịch vụ spring-boot phụ thuộc vào mongodb, do đó nó sẽ đợi cho tới khi container mongodb khởi chạy.
- **3. frontend:** Dịch vụ frontend phụ thuộc spring-boot, do đó nó được khởi chạy cuối cùng.

Chú ý: Theo mặc định, 'depends_on' chỉ đảm bảo container trước đã khởi chạy (process PID 1 chạy), chứ không đợi dịch vụ bên trong hoàn toàn sẵn sàng nhận kết nối. Để giải quyết, có thể dùng thuộc tính 'condition: service_healthy' kết hợp với Healthcheck.

3. Biểu diễn trực quan (Diagrams)

Dưới đây là các khoảng trống được thiết kế sẵn để dán sơ đồ trực quan. Hãy sử dụng các ảnh xuất ra từ Mermaid.js để dán trực tiếp vào các khung này.

[illegible]

Sơ đồ 2: Sơ đồ Kiến trúc (Component Diagram) - Điều phối trong Docker Daemon

